연결고리 개발 규약 — 물폭탄 대전 게임

Nov 5, 2026 · @영현

이 문서의 목적

서버·클라이언트가 주고받는 메시지와 팀 공통 규칙을 한 곳에 정해, 세 명이 서로 기다리지 않고 따로 개발한 뒤 바로 합칠 수 있게 한다. 이 문서는 1주차 안에 셋이 함께 확정한다.

- 이 문서의 내용과 코드가 다르면 **문서가 기준**이다. 코드를 문서에 맞추거나, 합의한 뒤 문서를 먼저 고친다.
- 메시지 규약이나 `common/` 패키지를 바꿀 때는 반드시 3인이 합의한다.
- 바꾼 내용은 맨 아래 [미정 사항] 체크리스트나 변경 기록에 한 줄로 남긴다.
- 아래 메시지 목록과 숫자는 **초안**이다. 팀 회의에서 고치고 확정한다.

공통 규칙

모든 게임 규칙은 서버가 판정하고, 클라이언트는 입력을 보내고 받은 상태를 그리기만 한다. 아래 값은 모두 `common/Constants.java`에 둔다.

항목	규칙 (초안)
맵 좌표	타일 단위 `(col, row)`, 왼쪽 위가 `(0, 0)`. 맵 크기 15×13 타일
플레이어 위치	픽셀 단위 `(x, y)` 정수. 타일 1칸 = 40px. 물폭탄은 플레이어 중심이 있는 타일에 설치
게임 시간	서버 틱 기준. 1틱 = 50ms(초당 20회). 폭탄 타이머 등은 틱 수로 계산
동시 판정	같은 틱에 일어난 일은 동시. 같은 틱에 마지막 생존자들이 모두 탈락하면 무승부
플레이어 ID	서버가 접속 순서대로 주는 정수 `userId`. 닉네임은 표시용
방 ID	서버가 주는 정수 `roomId`. 방 밖(로비)은 `0`
문자 인코딩	UTF-8. 소스 파일과 소켓 스트림 모두 `StandardCharsets.UTF_8` 명시
포트	`5000` (테스트는 `localhost`)
인원	방당 최소 2명, 최대 4명으로 시작 (4인 테스트 기준)

물폭탄 기본값도 같이 정해 둔다: 폭발까지 60틱(3초), 기본 위력 1칸, 기본 설치 개수 1개, 물방울에 갇히고 100틱(5초) 뒤 탈락.

메시지 규약

모든 메시지는 TCP 위에서 **길이(4바이트 int) + UTF-8 본문**으로 보낸다. 본문은 `|`로 필드를 나눈 문자열이라 로그로 읽기 쉽다.

프레임 형식

```
[int 길이][TYPE|roomId|userId|본문...]
```

- 보내기: 본문을 `getBytes(UTF_8)`로 바꾼 뒤 `writeInt(bytes.length)`, `write(bytes)`, `flush()` 순서로 보낸다.
- 받기: `readInt()`로 길이를 읽고 `readFully(buf)`로 그만큼 읽는다. `read` 한 번이 메시지 하나라고 가정하지 않는다.
- 클라이언트가 보내는 `roomId`, `userId`는 참고용이다. 서버는 그 소켓의 세션 정보를 기준으로 판단한다.
- 채팅처럼 `|`가 들어갈 수 있는 값은 항상 **마지막 필드**에 두고, `split("[|]", n)`처럼 나눌 개수를 제한한다.
- 목록은 `,`로, 항목 안의 값은 `:`로 나눈다.

클라이언트 → 서버

TYPE	본문	설명
`LOGIN`	`닉네임`	접속 후 첫 메시지
`ROOM_LIST`	(없음)	방 목록 요청
`ROOM_CREATE`	`방제목`	방을 만들고 방장으로 입장
`ROOM_JOIN`	`roomId`	방 입장
`ROOM_LEAVE`	(없음)	방 퇴장 (게임 중이면 탈락 처리)
`READY`	`true` 또는 `false`	준비 상태 변경
`START`	(없음)	방장만 보냄. 전원 준비 시 게임 시작
`INPUT`	`UP` / `DOWN` / `LEFT` / `RIGHT` / `STOP`	방향키를 누르거나 뗄 때만 보냄. 좌표는 보내지 않음
`BOMB`	(없음)	현재 위치에 물폭탄 설치 요청

TYPE	본문	설명
`CHAT`	`메시지`	같은 방 채팅
`EMOJI`	`이모티콘번호`	이모티콘 전송

서버 → 클라이언트

TYPE	본문	설명
`LOGIN_OK`	`userId`	접속 성공, ID 배정
`ERROR`	`코드:메시지`	요청 거절 (예: `ROOM_FULL:방이 가득 찼습니다`)
`ROOM_LIST`	`roomId:제목:인원:상태,...`	방 목록. 상태는 `WAIT` 또는 `PLAYING`
`ROOM_INFO`	`방장ID` · `userId:닉네임:준비,...`	필드 2개. 방 참가자가 바뀔 때마다 방 전체에 보냄
`GAME_INIT`	`맵문자열` · `userId:x:y,...`	필드 2개. 게임 시작. 맵은 타일 문자를 한 줄로 이은 것 (`.` 빈칸, `#` 벽, `B` 상자)
`STATE`	`tick` · `플레이어목록` · `폭탄목록`	필드 3개. 매 틱마다 방 전체에 보냄. 플레이어 `id:x:y:상태`, 폭탄 `col:row:남은틱`
`EVENT`	`종류:값...`	상태 외 변화. `EXPLODE:col:row:상:하:좌:우`, `BOX_BREAK:col:row:아이템`, `ITEM_GET:userId:col:row:종류`, `TRAPPED:userId`, `DEAD:userId`
`CHAT`	`userId:닉네임` · `메시지`	필드 2개. 채팅 중계
`EMOJI`	`userId:이모티콘번호`	이모티콘 중계
`PLAYER_LEFT`	`userId`	참가자가 나감
`RESULT`	`WIN:userId` 또는 `DRAW:userId,...`	게임 결과. 끝나면 방은 대기 상태로 돌아감

표에서 `·`로 구분한 것은 실제 메시지에서 `|`로 나뉘는 별도 필드다.

예시

```
C→S  INPUT|3|2|LEFT
S→C  STATE|3|0|418|1:200:120:ALIVE,2:360:80:TRAPPED|5:3:12
S→C  EVENT|3|0|EXPLODE:5:3:2:1:2:0
C→S  CHAT|3|2|안녕 | 반가워
```

`STATE` 는 1초에 20번씩 4명에게 보내므로 짧게 유지한다. 맵 전체는 `GAME_INIT` 에서 한 번만 보내고, 이후 변화는 `EVENT` 로만 알린다.

주요 흐름



메시지 흐름 · 접속에서 종료까지 4단계

게임 중에는 클라이언트가 입력만 보내고, 서버가 매 틱 확정된 `STATE` 를 방 전체에 보낸다. 연결이 끊기면 서버는 그 플레이어를 탈락 처리하고 `PLAYER_LEFT` 를 보낸 뒤 승패를 다시 판정한다.

파일 구조와 역할 분담

코드는 `common` / `server` / `client` 세 패키지로 나누고, 패키지별로 담당자를 정해 충돌을 줄인다. 담당자 이름은 팀 회의에서 채운다.

```
src/
├── common/
│   ├── protocol/   MsgType, Message, MessageCodec
│   ├── model/      Direction, TileType, PlayerState, ItemType
│   └── Constants.java
├── server/
│   ├── ServerMain.java
│   ├── net/        ClientHandler, Broadcaster
│   ├── lobby/      RoomManager, Room, UserSession
│   └── game/       GameLoop, GameState, GameMap, ServerPlayer, Bomb,
ExplosionCalc, Judge
└── client/
    ├── ClientMain.java
    ├── net/        Connection, ReceiverThread
    ├── ui/         MainFrame, LoginPanel, LobbyPanel, RoomPanel, ChatPanel,
ResultDialog
    ├── game/       GamePanel, InputHandler, ClientGameState, Animator
    └── resource/   ImageLoader
res/                images/, sounds/ (직접 만든 리소스만)
```

역할	담당 패키지	하는 일	담당자
게임 로직	`server/game`	게임 루프, 이동·충돌, 폭탄·연쇄 폭발, 피격·승패 판정, 아이템	
화면	`client/ui`, `client/game`	화면 전환, 게임 화면 그리기, 애니메이션, 키 입력	
네트워크·로비	`common/protocol`, `server/net`, `server/lobby`, `client/net`	메시지 규약, 접속·방·채팅, 접속 종료 처리	

- 네트워크 담당이 1주차에 `MsgType`과 `MessageCodec`을 먼저 올리면 나머지 두 명이 바로 시작할 수 있다.
- 서버 방 하나당 `GameLoop` 스레드 하나, 클라이언트 한 명당 `ClientHandler` 스레드 하나를 둔다.
- `ClientHandler`는 받은 입력을 바로 처리하지 않고 방의 `ConcurrentLinkedQueue`에 넣는다. `GameLoop`가 매 틱 꺼내 처리한다.
- 클라이언트의 `ReceiverThread`는 UI를 직접 건드리지 않고 `SwingUtilities.invokeLater`로 넘긴다.

협업 규칙

`main`은 항상 실행되는 상태로 유지하고, 2주차 안에 한 번 통합한다.

개발 환경

- IDE는 자유 (VS Code, IntelliJ, Eclipse). 대신 아래 규칙을 지킨다
- JDK 17, 소스 루트는 `src/`, 리소스는 `res/`. 어느 IDE든 프로젝트 루트를 작업 폴더로 실행한다
- 외부 라이브러리는 쓰지 않는다. 꼭 필요하면 `lib/`에 jar를 넣고 합의한다
- 인코딩 UTF-8 (Eclipse는 Workspace 기본값이 MS949일 수 있으니 반드시 바꾼다), 줄바꿈 LF, 들여쓰기 스페이스 4칸
- 이 설정은 루트의 `.editorconfig`와 `.gitattributes`(`* text=auto eol=lf`)로 Git에 올려 강제한다
- 각자 IDE가 만드는 설정 파일은 Git에 올리지 않는다. `.gitignore`: `bin/`, `out/`, `*.class`, `.idea/`, `*.iml`, `.project`, `.classpath`, `.settings/`, `.vscode/`
- 자동 포맷터로 남의 코드 전체를 재정렬하지 않는다. 내가 고친 줄만 커밋한다

Git

- 브랜치: 기능별로 `feature/lobby`, `feature/bomb`처럼 만든다
- 커밋 메시지: `[server] 폭탄 연쇄 폭발 구현`처럼 `[server]`, `[client]`, `[common]`, `[docs]` 중 하나로 시작
- `main`에는 PR로만 합치고, 다른 한 명이 훑어본 뒤 merge
- `common/` 수정은 작게, 자주 합친다. 합치기 전에 단톡방에 한 줄 알린다

기다리지 않고 개발하기

- 화면 담당은 고정된 `STATE`를 계속 보내는 **가짜 서버**로 개발한다
- 게임 로직 담당은 콘솔 출력과 간단한 **테스트 클라이언트**로 개발한다
- 4인 테스트는 한 PC에서 `Client`를 4번 실행해서 한다

통합 일정과 소통

시점	목표
1주차 끝	이 문서 확정, 서버 1개 + 클라이언트 4개 접속
2주차 끝	4명이 같은 방에 들어가 채팅하고, 네모 캐릭터가 움직이는 게 서로 보인다
매주	주 1~2회 짧은 진행 상황 공유, 막히면 단톡방에 바로 공유

미정 사항

팀 회의에서 정하고, 정해지면 체크한 뒤 위 본문에 반영한다.

- [ ] 역할별 담당자 확정
- [ ] 맵 크기(15×13), 타일 크기(40px), 틱 주기(50ms) 확정
- [ ] 물방울에 갇힌 플레이어를 다른 플레이어가 건드리면 즉시 탈락시킬지
- [ ] 상자에서 아이템이 나오는 확률과 아이템 종류별 효과
- [ ] 플레이어 이동 속도 (틱당 몇 px)
- [ ] 방 최대 인원을 4명으로 둘지, 그 이상 허용할지
- [ ] 방장이 나가면 다음 방장을 누구로 할지
- [ ] 에러 코드 목록 (`ROOM_FULL` , `NOT_HOST` , `NOT_READY` , `DUP_NICK` 등)
- [ ] 캐릭터·맵 이미지 제작 방식 (원작 리소스는 사용하지 않음)